

Rengar Prototype — техническая документация

Godot 4.3: боевые анимации, гуманоидный противник и компонентные зависимости

Автор: Manus AI

Дата: 27 августа 2026 г.

Содержание

1	Назначение и границы документа	1
1.1	Итог выполненного патча	1
1.2	Проверка	1
2	Быстрый запуск и управление	2
3	Файловая организация	2
3.1	Ресурсы Mixamo	3
4	Архитектура сцены	3
4.1	Отдельная карта зависимостей	4
5	Компоненты игрока	4
5.1	Машина анимаций игрока	4
6	Гуманоидный противник	5
6.1	Визуальная вариация	5
6.2	Контроллер анимаций противника	5
7	Карта сигналов и жизненный цикл удара	6
8	Как безопасно добавить следующий клип	7
8.1	Пример: будущий верхний удар	7
8.2	Почему не следует смешивать риги	7
9	Настройка и отладка	7
9.1	Параметры в Inspector	7
9.2	Диагностика	8
10	Известные ограничения и следующий безопасный шаг	8
11	Справочник терминов	8
12	Ссылки	9

1 Назначение и границы документа

Этот документ описывает фактическую структуру исправленного прототипа **Rengar Prototype** для Godot 4.3. Его задача — дать разработчику воспроизводимую картину сцены, компонентов, Mixamo-ресурсов, событий и порядка дальнейшего расширения. Документ основан на переданном исходном проекте, его сцене Актуально_22.08.26.tscn, актуальных GDScript-файлах и проверенном патче от 27 августа 2026 года.

> **Главный принцип патча:** физика, ИИ и расчёт урона остаются в существующих компонентах. Новые скрипты отвечают только за визуальное воспроизведение и оттенок модели. Благодаря этому боевая логика не дублируется между игроком и противником.

Для физического перемещения оба бойца используют `CharacterBody3D` и вызывают `move_and_slide()` в управляющих скриптах. Этот тип тела предназначен для управляемых скриптом персонажей и предоставляет высокоуровневое перемещение с обнаружением столкновений [1]. Анимационный слой отделён: `AnimationPlayer` хранит клипы, а `AnimationTree` обеспечивает расширенные переходы между ними [2] [3].

1.1 Итог выполненного патча

| Изменение | Где реализовано | Практический результат | |—|—| | Подключены `AT_LEFT.fbx` и `AT_RIGHT.fbx` | `MixamoAnimationLibrary.gd` | Оба клипа регистрируются как `AttackLeft` и `AttackRight` для модели с Mixamo-ригом. | | Добавлены состояния ударов игрока | Актуально_22.08.26.tscn, `MixamoAnimationComponent.gd` | Сигнал направленного удара переключает `PlayerAnimTree` в `AttackLeft` или `AttackRight`. | | Добавлен гуманоидный противник | Узел `EnemyVisual` | Куб заменён на экземпляр `Y Bot.fbx`; у игрока и противника используется совместимый скелет. | | Добавлен фиолетовый внешний вид | `ModelTintComponent.gd` | На все доступные сетки `EnemyVisual` назначается единый фиолетовый `material_override`. | | Добавлены анимации противника | `EnemyAnimationComponent.gd` | Враг воспроизводит `Idle`, `Run`, `AttackLeft`, `AttackRight` и `HitReact`, не изменяя ИИ. | | Защищён эффект попадания | `EnemyController.gd` | Вспышка применяется к сеткам гуманоидной модели и затем возвращает исходный фиолетовый материал. |

1.2 Проверка

Патч проверен в Godot 4.3.stable.official.77dcf97d8 под виртуальным графическим дисплеем с OpenGL. Автоматический smoke-тест загружал главную сцену, удалял влияние ИИ на тест, вручную отпирал `attack_performed("LEFT")` и `attack_performed("RIGHT")` игроку и противнику и получил маркер `ANIMATION_SMOKE_TEST: PASS`. В логе два загрузчика подтвердили наличие обоих клипов: `AttackLeft` и `AttackRight`.

| Проверка | Результат | Что именно подтверждено | |—|—| | Импорт FBX | Успешно | `AT_LEFT.fbx` и `AT_RIGHT.fbx` импортируются как сцены Godot. | | Загрузка сцены | Успешно | Главная сцена создаётся без ошибок разбора GDScript или .tscn. | | Игрок: левый удар | Успешно | `PlayerAnimTree` достигает `AttackLeft`. | | Игрок: правый удар | Успешно | `PlayerAnimTree` достигает `AttackRight` после короткого cross-fade. | | Враг: левый и правый удар | Успешно | Его `AnimationPlayer.current_animation` становится `AttackLeft` и `AttackRight`. | | Два экземпляра модели | Успешно | Библиотека Mixamo инициализируется отдельно у `VisualModel` и `EnemyVisual`. |

2 Быстрый запуск и управление

Открывайте `project.godot` через Godot 4.3 и запускайте главную сцену `Актуально_22.08.26.tscn`. Главная сцена задана в `project.godot` как `run/main_scene`. В проекте нет отдельного меню, сохранений или загрузки уровней: это компактный технический прототип одной тестовой арены.

| Действие | Ввод | Источник обработки | Визуальная реакция | |—|—|—| | Движение | W, A, S, D |
 Player3d.gd → StateComponent.gd | Blend Idle/Run в Motion. | | Прыжок | Space | Player3d.gd → StateComponent.gd | Состояние Jump, пока тело не на полу. | | Направленный удар | Удержание и отпускание ЛКМ с движением мыши | Player3d.gd → CombatComponent.gd | Влево — AttackLeft; вправо — AttackRight; вверх — временный fallback AttackRight. | | Блок | Удержание ~ и движение мыши | CombatComponent.gd | Игровой блок действует; отдельные block-клипы не входят в данный патч. | | Dash / dodge | Существующие комбинации проекта | StateComponent.gd | Физическое действие сохраняется; для него не добавлялись новые Mixamo-клипы. |

Направление удара вычисляется не по клавиатуре, а по накопленной дельте мыши во время замаха. CombatComponent выдаёт строку UP, DOWN, LEFT или RIGHT через сигнал `attack_performed(direction, is_leap_attack)`. В рамках предоставленного набора существуют только два новых клипа. Поэтому LEFT однозначно сопоставлен с AttackLeft, RIGHT — с AttackRight, а UP и DOWN используют AttackRight как явно задокументированный временный fallback. Это сохраняет визуальный отклик для любого подтверждённого удара, но не выдаёт его за отдельную верхнюю или нижнюю анимацию.

3 Файловая организация

Ниже приведена рабочая часть дерева. Временные файлы редактора `.godot/` намеренно не входят в итоговую поставку: Godot пересоздаёт их при первом открытии проекта.

```
Rengar_Prototype_AnimationPatch/
├─ project.godot
├─ Актуально_22.08.26.tscn
├─ Assets/
│   └─ Mixamo/
│       ├── Y Bot.fbx           # базовая humanoid-модель и риг
│       ├── Idle.fbx
│       ├── Running.fbx
│       ├── Running Jump.fbx
│       ├── Reaction.fbx
│       ├── AT_LEFT.fbx        # новый левый удар
│       └─ AT_RIGHT.fbx       # новый правый удар
└─ Scripts/
    ├── Player3d.gd
    ├── EnemyController.gd
    ├── StateComponent.gd
    └─ CombatComponent.gd
```

```

└─ HealthComponent.gd
└─ StacksComponent.gd
└─ MixamoAnimationLibrary.gd      # расширен
└─ MixamoAnimationComponent.gd    # расширен
└─ EnemyAnimationComponent.gd     # новый
└─ ModelTintComponent.gd         # новый

```

3.1 Ресурсы Mixamo

Все клипы должны иметь совместимый Mixamo-скелет с Y Bot.fbx. Файлы клипов могут содержать дополнительную геометрию, но в игре отображается только один экземпляр базовой модели. MixamoAnimationLibrary загружает сцену-клип, находит в ней AnimationPlayer, дублирует анимацию mixamo_com и помещает копию в AnimationPlayer нужной видимой модели.

| Игровое имя | FBX-источник | Loop | Потребитель | Назначение | |---|---|---| | Idle | Idle.fbx | Да | Игрок, враг | Неподвижная стойка. | | Run | Running.fbx | Да | Игрок, враг | Цикл движения; предполагается загрузка In Place. | | Jump | Running Jump.fbx | Нет | Игрок | Прыжок и прыжок из куста. | | HitReact | Reaction.fbx | Нет | Игрок, враг | Реакция на полученный урон или стан. | | AttackLeft | AT_LEFT.fbx | Нет | Игрок, враг | Левый направленный удар. | | AttackRight | AT_RIGHT.fbx | Нет | Игрок, враг | Правый направленный удар и текущий fallback. |

4 Архитектура сцены

Главная сцена сохраняет простой корень World. Внутри находятся камера, два CharacterBody3D, зона куста, свет, поверхность земли и PlayerUI. Визуальная модель — это дочерний экземпляр тела, а не объект, управляющий перемещением. Такое разделение важно: анимация меняет позу костей, а положение на уровне мира меняется только через CharacterBody3D.velocity и move_and_slide().

```

World
└─ Camera3D                      (ThirdPersonCamera.gd)
└─ PLayerCharacterBody3D        [группа player]
  └─ CollisionShape3D
  └─ VisualModel                (Y Bot.fbx + MixamoAnimationLibrary.gd)
  └─ HealthComponent
  └─ CombatComponent
  └─ StateComponent
  └─ StacksComponent
  └─ AnimationComponent          (MixamoAnimationComponent.gd)
  └─ PlayerAnimTree
└─ EnemyCharacterBody3D         [группа enemy]
  └─ CollisionShape3D
  └─ EnemyVisual                (Y Bot.fbx + MixamoAnimationLibrary.gd)
  └─ ModelTintComponent

```

```

|   └─ HealthComponent
|   └─ CombatComponent
|   └─ StateComponent
|   └─ AnimationComponent      (EnemyAnimationComponent.gd)
└─ Bush                        [группа bush]
└─ DirectionalLight3D
└─ PlayerUI
└─ Ground

```

4.1 Отдельная карта зависимостей

К этому пакету приложены `dependency_tree.png` и исходник `dependency_tree.mmd`. PNG — широкая фрактальная карта связей между сценой, общими FBX-ресурсами, компонентами и сигналами. Открывайте её отдельно с масштабом 100% или выше: тогда читаются имена всех узлов и цветовые группы. Mermaid-исходник нужен, чтобы карту можно было обновлять вместе с кодом, а не поддерживать вручную как растровое изображение.

5 Компоненты игрока

`Player3d.gd` — тонкий оркестратор. Он принимает ввод, передаёт его в `StateComponent` и `CombatComponent`, а после изменения `velocity` вызывает `move_and_slide()`. Скрипт не хранит здоровье, не строит хитбоксы и не знает деталей Mixamo-клипов.

Компонент	Ответственность	Зависимости	Чего компонент не делает
<code>HealthComponent.gd</code>	Хранит <code>hp</code> , <code>max_hp</code> , состояние смерти; посылает <code>hp_changed</code> , <code>died</code> .	Родитель-тело.	Не показывает UI и не воспроизводит анимации.
<code>CombatComponent.gd</code>	Замах, выбор направления, блок, хитбоксы, урон, таймеры и состояние врага.	<code>HealthComponent</code> , <code>StateComponent</code> , группы <code>player/enemy</code> .	Не переключает клипы напрямую.
<code>StateComponent.gd</code>	Обычное движение, гравитация, прыжок, стелс, <code>leap</code> , <code>dash</code> и <code>dodge</code> .	Родитель <code>CharacterBody3D</code> .	Не меняет здоровье и не создаёт боевой хитбокс.
<code>StacksComponent.gd</code>	Счётчик стэков игрока и сигналы готовности усиления.	Игрок и UI.	Не знает об анимациях противника.
<code>MixamoAnimationComponent.gd</code>	Синхронизирует визуальный <code>AnimationTree</code> с движением, прыжком, уроном и направленным ударом.	<code>PlayerAnimTree</code> , <code>VisualModel/AnimationPlayer</code> , <code>CombatComponent</code> , <code>StateComponent</code> .	Не двигает тело и не применяет урон.

5.1 Машина анимаций игрока

`PlayerAnimTree` использует `VisualModel/AnimationPlayer` как источник клипов. У `AnimationTree` в Godot назначается путь к `AnimationPlayer`, а воспроизведение и переходы должны управляться через само дерево и его `AnimationNode`, а не параллельно через тот же `AnimationPlayer` [3]. Поэтому игроком управляет `MixamoAnimationComponent`, который получает `AnimationNodeStateMachinePlayback` из `parameters/playback`.

Состояние	Вход	Выход	Содержимое
<code>Motion</code>	Запуск сцены, окончание реакций и атак	Прыжок, урон, удар	<code>BlendSpace1D</code> : Idle при нулевой скорости и Run при скорости.
<code>Jump</code>	Тело не на полу или сигнал <code>state_changed("leap", true)</code>	Приземление	<code>Jump</code> .
<code>HitReact</code>			

CombatComponent.damage_taken | Таймер 0.35 с | HitReact. | | AttackLeft | attack_performed("LEFT")
 | Таймер длины клипа | AttackLeft. | | AttackRight | attack_performed("RIGHT"), временный UP/
 DOWN fallback | Таймер длины клипа | AttackRight. |

Код получает фактическую длину анимации из AnimationPlayer.get_animation(clip_name).length, вычитает 0.06 с для возврата немного до последнего кадра и использует резерв attack_recovery_fallback = 0.72, если ресурс недоступен. При смене направления применяется root_playback.travel(clip_name, true), а при повторе того же направления — start(clip_name, true). Между AttackLeft и AttackRight добавлены прямые переходы с cross-fade 0.04 с; так быстрый сменный удар не обязан сначала возвращаться в Motion.

```
func _on_attack_performed(direction: String, _is_leap_attack: bool) -> void:
    var clip_name: StringName = &"AttackRight"
    if direction == "LEFT":
        clip_name = &"AttackLeft"
    elif direction == "RIGHT":
        clip_name = &"AttackRight"
    _play_attack(clip_name)
```

6 Гуманоидный противник

Куб EnemyVisual заменён на экземпляр того же Y Bot.fbx, что применяется для игрока. На корень противника не переносится меш, кости или логика игрока: он получает собственный экземпляр сцены модели и собственную копию библиотеки анимаций. Это исключает зависимость позы противника от PlayerAnimTree и не создаёт второго тела управления.

6.1 Визуальная вариация

ModelTintComponent.gd ищет все MeshInstance3D внутри EnemyVisual и назначает им один StandardMaterial3D:

```
@export var tint_color := Color(0.44, 0.12, 0.72, 1.0)
@export var roughness := 0.72

for mesh_node in meshes:
    var mesh := mesh_node as MeshInstance3D
    if mesh != null and mesh.mesh != null:
        mesh.material_override = tint_material
```

Оттенок RGB соответствует глубокому фиолетовому. Проверка mesh.mesh != null пропускает возможные импортированные вспомогательные узлы, которые не имеют геометрии. Таким образом, визуальная настройка не влияет на Skeleton3D, коллайдер, слои столкновений, здоровье или ИИ.

6.2 Контроллер анимаций противника

EnemyAnimationComponent.gd — отдельный визуальный компонент. Он слушает только сигналы уже существующего CombatComponent и читает горизонтальную скорость CharacterBody3D. У

противника нет `AnimationTree`: для минимальности он вызывает `AnimationPlayer.play()` у собственного `EnemyVisual/AnimationPlayer`. Это допустимо, потому что этот `AnimationPlayer` не связан с `AnimationTree` противника.

| Условие | Выбор клипа | Источник | |—|—|—| | Горизонтальная скорость больше 0.15 | Run | `owner_body.velocity.` | | Скорость не превышает порог | Idle | `owner_body.velocity.` | | `attack_performed("LEFT")` | AttackLeft | CombatComponent врага. | | `attack_performed("RIGHT")` | AttackRight | CombatComponent врага. | | `attack_performed("UP")` | AttackRight (fallback) | В текущем наборе нет верхнего клипа. | | `damage_taken` или `stun_started` | HitReact | CombatComponent врага. | В `EnemyController.gd` ИИ по-прежнему выбирает направление и создаёт хитбокс. При получении урона контроллер сначала запускает цветовую вспышку, затем отображает число урона, звук и частицы, изменяет здоровье и вызывает существующее отбрасывание. Патч изменяет только способ поиска мешей для вспышки: теперь ищутся рендеримые дочерние сетки `EnemyVisual`, их текущие `material_override` сохраняются, на 0.15 с ставится бело-красный материал и затем восстанавливается фиолетовый `override`.

7 Карта сигналов и жизненный цикл удара

Следующая последовательность — ключ к тому, почему визуал не дублирует игровую логику.

```

ЛКМ удержана + движение мыши
    ↓
Player3d.gd вызывает CombatComponent.start_attack()
    ↓
При отпускании ЛКМ CombatComponent определяет UP/DOWN/LEFT/RIGHT
    ↓
attack_performed(direction, is_leap_attack)
    ├── MixamoAnimationComponent выбирает визуальный клип игрока
    └── _spawn_player_hitbox() создаёт временный Area3D
        ↓
        body.take_dmg(...) у цели
            ↓
            CombatComponent.damage_taken или enemy_on_take_dmg(...)
                ↓
                HitReact / вспышка / HP / отбрасывание
  
```

| Сигнал | Отправитель | Получатель | Назначение | |—|—|—| | `attack_performed(direction, is_leap_attack)` | CombatComponent | MixamoAnimationComponent, EnemyAnimationComponent | Выбирает визуальное направление; создание хитбокса происходит в том же боевом компоненте. | | `damage_taken` | CombatComponent | MixamoAnimationComponent, EnemyAnimationComponent | Включает HitReact. | | `stun_started(duration)` | CombatComponent | EnemyAnimationComponent | Проигрывает HitReact на время стана. | | `state_changed("leap", value)` | StateComponent | MixamoAnimationComponent | Переключает игрока на Jump. | | `hp_changed, died` | HealthComponent | EnemyController, UI | Обновляет индикаторы; для врага смерть удаляет узел. |

8 Как безопасно добавить следующий клип

Пайплайн остаётся преднамеренно коротким. Новый FBX кладётся в Assets/Mixamo/; затем его регистрируют ровно в одном месте — словаре ANIMATION_SOURCES внутри MixamoAnimationLibrary.gd. Не нужно создавать второй Skeleton3D, перетаскивать десятки tracks вручную или менять CombatComponent только ради визуала.

8.1 Пример: будущий верхний удар

1. В Mixamo выбрать **того же Y Bot**, открыть подходящую верхнюю атаку и скачать FBX на 30 FPS. Для атаки включить **In Place**, если переключатель доступен; иначе root-motion может конкурировать с физическим перемещением CharacterBody3D.
2. Сохранить файл как Assets/Mixamo/AT_UP.fbx.
3. В MixamoAnimationLibrary.gd добавить &"AttackUp": "res://Assets/Mixamo/AT_UP.fbx",.
4. В .tscn добавить AnimationNodeAnimation с animation = &"AttackUp", состояние AttackUp и переходы из/в Motion.
5. В on_attack_performed сопоставить direction == "UP" с &"AttackUp" вместо текущего fallback.
6. Добавить AttackUp в выбор EnemyAnimationComponent, если ИИ продолжает генерировать UP.
7. Перезапустить сцену и проверить лог MixamoAnimationLibrary: загружены [...]; затем убедиться, что клип играет на базовой модели до настройки переходов.

```
const ANIMATION_SOURCES := {
    &"AttackUp": "res://Assets/Mixamo/AT_UP.fbx",
}

if direction == "UP":
    clip_name = &"AttackUp"
```

8.2 Почему не следует смешивать риги

Копируются только треки клипа, поэтому имена костей и rest pose должны быть совместимы с видимой моделью. Практическое правило для этого прототипа: все новые клипы нужно скачивать после выбора одного и того же персонажа в Mixamo. Если конечная модель Ренгара будет иметь другой rig, правильный следующий шаг — ретаргетинг в DCC-инструменте или отдельный слой импорта, а не случайная замена осей в AnimationTree.

9 Настройка и отладка

9.1 Параметры в Inspector

| Узел | Поле | Значение по умолчанию | Эффект | |---|---| | Игрок → AnimationComponent | run_speed_for_full_blend | 6.0 | Скорость, при которой Motion полностью переходит к Run. | | Игрок → AnimationComponent | attack_recovery_fallback | 0.72 | Запасная длительность удара, если длина клипа недоступна. | | Впар → AnimationComponent | run_speed_for_full_animation | 0.15 | Порог перехода с Idle на Run. | | Впар → AnimationComponent | fallback_attack_duration | 0.72 | Запасная блокировка переключения во время удара. | | Впар → ModelTintComponent | tint_color |

(0.44, 0.12, 0.72, 1.0) | Основной фиолетовый оттенок врага. | | Враг → ModelTintComponent | roughness | 0.72 | Матовая поверхность, уменьшающая слишком яркие блики. |

9.2 Диагностика

| Симптом | Вероятная причина | Действие | |—|—| | В логе нет AttackLeft/AttackRight | Неверный путь в ANIMATION_SOURCES или FBX не импортирован. | Сверить регистр имени и путь res:// Assets/Mixamo/...; открыть FBX в Godot и проверить AnimationPlayer. | | Появляется предупреждение «нет клипа mixamo_com» | Импортер дал другому клипу другое имя. | Открыть FBX, определить фактическое имя в AnimationPlayer и заменить строку поиска в _add_clip(). | | Модель делает шаг по миру во время удара или бега | Клип содержит root motion. | Скачивать с **In Place**; мировую позицию оставлять CharacterBody3D. | | Игрок остаётся в прошлой атаке | Нет перехода к новому состоянию или клип не зарегистрирован. | Проверить AttackLeft/AttackRight в дереве, прямые переходы и лог загрузчика. | | Противник выглядит серым | ModelTintComponent не нашёл рендеримую сетку. | Убедиться, что EnemyVisual — экземпляр Y Bot и путь компонента равен ../EnemyVisual. | | После попадания враг потерял фиолетовый цвет | Старый эффект попадания сбрасывает материал. | Использовать текущий EnemyController.gd, который сохраняет и восстанавливает material_override. | | Враг не играет удар | EnemyAnimationComponent не видит AnimationPlayer либо CombatComponent не испускает сигнал. | Проверить дочерние имена EnemyVisual, CombatComponent и наличие клипа в выводе. |

10 Известные ограничения и следующий безопасный шаг

Патч сознательно не превращает ранний прототип в полную боевую систему. В нём нет новых block-, dodge-, dash-, kick- или верхних attack-клипов, нет ретаргетинга финального персонажа Ренгара и нет отдельной боевой машины состояний для сложных комбо. Существующая архитектура также содержит старый AnimationComponent.gd и узел PlayerAnim, оставшиеся от ранней версии дерева; активным контроллером игрока в главной сцене является MixamoAnimationComponent.gd. | Приоритет | Следующий шаг | Причина | |—|—| | P0 | Скачать AttackUp и направленный kick-клип на том же Y Bot. | Устранит временный fallback верхнего/нижнего удара. | | P1 | Добавить реальные клипы block/dodge/dash. | Синхронизирует уже существующую физическую механику с визуалом. | | P1 | Вынести игрока и врага в самостоятельные .tscn. | Повторное использование станет проще без усложнения компонентов. | | P2 | Заменить Y Bot финальными моделями после ретаргетинга. | Сохраняет кодовую архитектуру, но требует совместимого rig. | | P2 | Добавить тесты на hitbox timing и реакции. | Свяжет момент урона с кадрами замаха, а не только с фактом проигрывания клипа. |

11 Справочник терминов

Ниже приведён печатный справочник используемых терминов. Он формируется из единого списка glossary-entries, поэтому определения можно расширять без расхождения между ссылками и итоговой терминологией.

AnimationPlayer: Контейнер библиотек и проигрыватель клипов.

AnimationTree: Дерево переходов и смешивания состояний игрока.

CharacterBody3D: Физический корень бойца, который хранит velocity и перемещается через move_and_slide().

Компонент: Дочерний узел с одной областью ответственности: здоровье, бой, состояние или визуальная анимация.

Fallback: Явно временная подстановка существующего клипа, пока целевой клип не добавлен.

Hitbox: Временный Area3D, который создаёт CombatComponent для обнаружения цели удара.

material_override: Материал поверх материалов меша; используется для фиолетового врага и краткой вспышки попадания.

MixamoAnimationLibrary: Локальный загрузчик, копирующий клипы из отдельных FBX в AnimationPlayer отображаемой модели.

Root motion: Перемещение, записанное в анимации; в данном прототипе его следует избегать через In Place.

Skeleton3D: Набор костей, на который назначаются треки FBX-анимации.

12 Ссылки

- [1] [Godot Engine — CharacterBody3D documentation.](#)
- [2] [Godot Engine 4.3 — AnimationPlayer documentation.](#)
- [3] [Godot Engine 4.3 — AnimationTree documentation.](#)
- [4] [Godot Engine 4.3 — Using AnimationTree.](#)